

URL Shortener Web Application

Internship Project – **Innomatics Research Labs**

1. Introduction

In today's digital world, URLs are widely used to share resources such as websites, documents, and media. However, many URLs are long and complex, making them difficult to share, copy, or remember. This is where a URL shortener becomes useful. A URL shortener converts a long URL into a shorter and more manageable form, which can then be easily shared and accessed.

The objective of this project is to design and develop a basic URL Shortener Web Application using Flask. The application allows users to enter a long URL, generate a shortened version, store it in a database, and later access previously shortened URLs. This project also focuses on validating user input and using a database ORM for data management.

This project was developed as part of my internship learning at Innomatics Research Labs and helped me gain hands-on experience with backend development, databases, and web application workflows.

2. Objectives of the Project

The main objectives of this project are:

- To create a web application that shortens long URLs.
- To validate whether the URL entered by the user is correct.
- To store original URLs and shortened URLs in a database.
- To redirect users from a shortened URL to the original URL.
- To maintain a history of all URLs shortened by the user.
- To implement the project using Flask and a database ORM.

3. Technology Stack Used

The following technologies were used to develop the project:

Backend

Python: Core programming language.

Flask: Lightweight web framework used to build the web application.

Frontend

HTML: Used to create the user interface of the application.

Database

SQLite: Lightweight database used for storing URLs.

SQLAlchemy: ORM (Object Relational Mapper) used to interact with the database.

Additional Libraries validators: Used to validate whether the entered URL is correct or not.

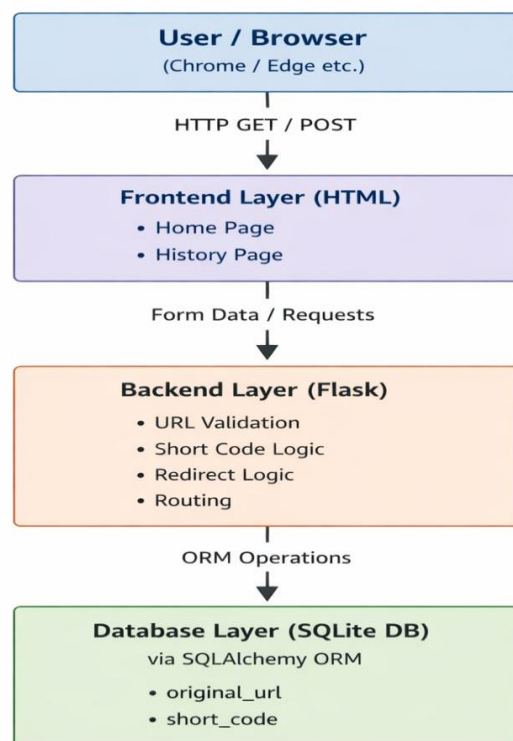
3. System Architecture

The application follows a three-layer architecture:

Frontend (HTML) → Backend (Flask) → Database (SQLite via SQLAlchemy ORM)

The architecture of the URL Shortener Web Application is divided into three layers. The user interacts with the application through a web browser. The frontend is developed using HTML and provides pages for URL input and viewing URL history. The frontend communicates with the backend using HTTP requests

The backend is implemented using Flask and is responsible for URL validation, short code generation, routing, and redirection logic. Database operations are handled using SQLAlchemy ORM, which communicates with an SQLite database for storing original URLs and their corresponding shortened codes



4. Project Workflow

The workflow of the URL Shortener Web Application is as follows:

- The user opens the home page of the application.

- The user enters a long URL into the input field.
- When the user clicks the “Shorten URL” button, the form data is sent to the backend.
- The backend validates whether the entered URL is valid.
- If the URL is invalid, an error message is displayed to the user.
- If the URL is valid, a random short code is generated.
- The original URL and its short code are saved into the database.
- The shortened URL is displayed on the home page.
- When the user clicks the shortened URL, the application redirects the user to the original URL.
- The user can view all previously shortened URLs on the history page.

6. Database Design and ORM Usage

The database contains a single table to store URL information. The table includes the following fields:

- **id:** A unique identifier for each record.
- **original_url:** The long URL entered by the user.
- **short_code:** The generated short code corresponding to the original URL.

SQLAlchemy ORM is used to define the database model and perform operations such as inserting records and querying data. Using ORM simplifies database interaction and avoids writing raw SQL queries.

7. URL Validation

- The application uses the validators library to verify whether the entered URL is valid.
- URL validation is performed before generating the short URL and saving it to the database.
- Invalid or malformed URLs are rejected and an appropriate error message is shown to the user.
- This validation step ensures data accuracy and prevents incorrect URLs from being stored in the system.

8. Features Implemented

- The following features were successfully implemented in the project:
- URL shortening functionality.
- URL validation before saving.
- Database storage using ORM.
- Redirection from short URL to original URL.
- History page displaying all shortened URLs.
- Clean and simple user interface.

9. Challenges Faced

- During the development of this project, I faced several challenges:
- Understanding how Flask handles routing and dynamic URLs.
- Implementing redirection using short URL codes.
- Ensuring that the generated short codes are unique.
- Integrating SQLAlchemy ORM with Flask.
- Validating user input correctly.

These challenges helped me improve my problem-solving skills and understanding of backend development.

10. Conclusion

The URL Shortener Web Application was successfully developed using Flask, HTML, and SQLAlchemy ORM. The project fulfills all the given requirements, including URL shortening, validation, database storage, redirection, and history tracking.

This project provided valuable hands-on experience in building real-world web applications and understanding how frontend, backend, and databases work together. It also strengthened my understanding of Flask routing, ORM usage, and input validation.

.

11. Future Enhancements

The application can be enhanced further by:

- Adding user authentication.
- Improving the UI with CSS and JavaScript.
- Preventing duplicate URLs from generating multiple short codes.
- Deploying the application to a cloud platform.
- Adding analytics for URL clicks.